Contenido

Aprendizaje no supervisado: Clustering 2

KMEANS 5

DBSCAN 10

Aplicación práctica del aprendizaje no supervisado. 14

Optimización de modelos de clustering 16

Validación en aprendizaje no supervisado 17

Aprendizaje no supervisado: Clustering .

Vamos a estudiar un conjunto de técnicas de **aprendizaje no supervisado**, es decir, técnicas que trabajan con **datos sin etiquetar**. Dentro de este tipo de aprendizaje destacan los llamados **algoritmos de clustering**, cuyo objetivo principal es **agrupar los datos en subconjuntos en función de su similitud**.

Estos algoritmos permiten organizar un conjunto de datos sin etiquetar en grupos coherentes, basándose en **criterios matemáticos**. De este modo, ayudan al analista a **descubrir estructuras internas en los datos** que, a simple vista, no son evidentes.

A estas agrupaciones se les denomina **clústeres**.

Un **clúster** es simplemente un conjunto de datos que comparten ciertas características y que, por tanto, son **más similares entre sí que con otros datos del conjunto**.

Por ejemplo, si tenemos un conjunto de registros de red, un clúster podría agrupar conexiones que presentan un comportamiento similar.

Agrupamiento de los datos

Para poder agrupar los datos, es necesario definir **reglas que permitan medir la similitud entre ellos**. Aquí aparece un concepto fundamental: **La proximidad o distancia entre los datos.**

Los algoritmos de clustering utilizan una **métrica de proximidad**, que es una forma de medir qué tan "cerca" o "lejos" están dos ejemplos entre sí.

- Si dos datos están muy cerca → probablemente pertenezcan al mismo clúster.
- Si están lejos → probablemente pertenezcan a clústeres distintos.

Es importante destacar que la forma de medir esa distancia depende del algoritmo que utilicemos.

<u>Ejemplo sencillo.</u>

Antes de ver algoritmos reales, vamos a ver un ejemplo muy básico que nos ayudará a entender la idea de agrupación.

Paso 1: Elegir una métrica de proximidad.

Lo primero que necesitamos es decidir **cómo vamos a medir la similitud entre los datos**. Vamos a utilizar una métrica muy sencilla:

Agrupar datos que compartan el mismo valor en una determinada característica.

Paso 2: Aplicarlo a un caso real.

Imaginemos que tenemos un conjunto de **logs de un servidor web (por ejemplo, Apache)**. De cada log podemos extraer información como:

- Dirección IP
- Método HTTP (GET, POST…)

- URL solicitada
- etc.

Esto se puede representar como un conjunto de variables: x1, x2, x3, …, xn

Paso 3: Definir la regla de agrupación.

Supongamos que elegimos una sola dimensión: la **dirección IP**. La regla sería:

- Todos los logs que tengan la **misma IP** → se agrupan en un clúster.
- Los logs con IP distinta → se agrupan en otros clústeres.

Interpretación.

Lo que estamos haciendo es definir una métrica muy simple: "Dos datos son similares si tienen la misma IP."

Este ejemplo, aunque sencillo, nos permite entender la idea fundamental del clustering:

Los algoritmos agrupan datos en función de una medida de similitud, creando estructuras (clústeres) que nos ayudan a interpretar y analizar conjuntos de datos sin etiquetar.

Ejemplo gráfico:

Vamos a analizar ahora un ejemplo visual para entender mejor cómo funciona el aprendizaje no supervisado. En este tipo de problemas, nuestro conjunto de datos **cambia respecto al aprendizaje supervisado**. En lugar de tener datos del tipo: *(x1, x2, y),* donde **y es la etiqueta o resultado esperado**, en aprendizaje no supervisado trabajamos únicamente con: *(x1, x2).* Es decir, **solo disponemos de las variables de entrada**, pero **no conocemos la salida**.

¿Qué significa esto en la práctica? Todos los datos que tenemos aparecen representados (por ejemplo, en una gráfica) como puntos, pero: **No sabemos a qué clase pertenece cada uno.**

Por eso, en muchas representaciones, todos los puntos aparecen del mismo color (por ejemplo, en negro), ya que no podemos distinguir si pertenecen a una clase u otra.

Ejemplo aplicado a redes.

Imaginemos que estamos trabajando con **logs de un servidor web (por ejemplo, Apache)**.

Cada línea de log representa una petición, y queremos resolver el siguiente problema determinar si una petición es: legítima o maliciosa.

El problema es que: **NO sabemos cuáles son legítimas y cuáles son maliciosas.** Ya que no tenemos etiquetas ni la variable y como ocurría en aprendizaje supervisado.

¿Qué hacemos entonces? Le damos al algoritmo: Todas las peticiones (logs) sin ninguna clasificación previa. Y dejamos que el modelo trabaje.

Lo que esperamos es que el algoritmo agrupe automáticamente las peticiones en diferentes clústeres.

Por ejemplo:

- Un grupo podría contener tráfico normal.
- Otro grupo podría contener tráfico sospechoso.

El algoritmo no nos dice directamente: “esto es un ataque” o “esto es normal”. Pero sí nos proporciona algo muy valioso: **una estructura de los datos.**

Y gracias a esa estructura, nosotros como analistas podemos:

- Identificar patrones.
- Interpretar los grupos.
- Deducir qué clúster corresponde a tráfico legítimo o malicioso.

En aprendizaje no supervisado, el algoritmo no conoce la respuesta, pero nos ayuda a descubrirla organizando los datos en grupos con significado.

Por ejemplo, imaginaros que en este caso x2 se va a corresponder con una característica determinada que extraemos de esas líneas de log y x1 se va a corresponder con otra característica determinada que extraemos de esas líneas de logs.

Si extraemos las características de nuestros datos y las representamos en un plano, podemos obtener una gráfica como la anterior. En este caso, por ejemplo, podemos tener:

- **x2 en el eje horizontal.**
- **x1 en el eje vertical.**

Aunque el orden de los ejes no es especialmente relevante, esta representación nos permite visualizar cómo se distribuyen los datos.

¿Qué hace un algoritmo de clustering en este caso?

Un algoritmo de clustering analiza este conjunto de puntos y trata de **identificar agrupaciones naturales** dentro de los datos. Es decir, su objetivo es:

Dividir los ejemplos del conjunto de entrenamiento en **subconjuntos o clústeres** en función de su similitud.

Al aplicar el algoritmo, podríamos obtener una separación como la siguiente:

- Un clúster con los puntos situados a la izquierda.
- Otro clúster con los puntos situados a la derecha.

¿Por qué se produce esta agrupación? La razón es el concepto de **proximidad**. El algoritmo detecta que:

- Los puntos de la izquierda están **cerca entre sí.**
- Los puntos de la derecha también están **cerca entre sí.**
- Pero ambos grupos están **separados entre sí.**

A partir de esta información, el algoritmo "deduce" que: El conjunto de datos está formado por **dos grupos principales.**

El algoritmo no sabe qué representa cada grupo, pero sí es capaz de detectar que existen dos conjuntos de datos con comportamientos diferentes basándose únicamente en la proximidad entre los puntos.

KMEANS

Una vez que hemos entendido el concepto general de **clustering**, vamos a centrarnos en una de las técnicas más importantes y utilizadas dentro del aprendizaje no supervisado: el algoritmo **K-Means**.

K-Means es uno de los algoritmos de clustering más conocidos y utilizados en Machine Learning. Su popularidad se debe principalmente a que:

- Es **sencillo de entender.**
- Es **fácil de implementar.**
- Funciona de manera eficiente incluso con **grandes volúmenes de datos.**

En muchos problemas reales, especialmente en entornos como redes o análisis de datos masivos, trabajamos con:

- Grandes cantidades de datos.
- Sin etiquetar.

Por tanto, necesitamos algoritmos que:

- Sean rápidos.
- Escalen bien.
- No requieran demasiados recursos.

K-Means cumple perfectamente con estas características.

Tipo de aprendizaje

K-Means es un algoritmo de **aprendizaje no supervisado**, lo que significa que:

- Recibe un conjunto de datos sin etiquetas.
- No conoce la respuesta correcta.
- Su objetivo es **descubrir estructuras en los datos.**

Como ya hemos visto en clustering, todos los algoritmos necesitan una forma de medir la similitud entre los datos.

En el caso de K-Means, se utiliza una métrica muy concreta: **La distancia euclídea.**

¿Qué es la distancia euclídea?

Es la distancia "normal" entre dos puntos en un plano. Por ejemplo, si tienes dos puntos:

- A = (x1, y1).
- B = (x2, y2).

La distancia es la longitud de la línea recta entre ellos.

Cuanto menor es la distancia → más similares son los datos.
Cuanto mayor es la distancia → más diferentes son.

K-Means utiliza esta idea de distancia para agrupar los datos en función de qué tan cerca están unos de otros

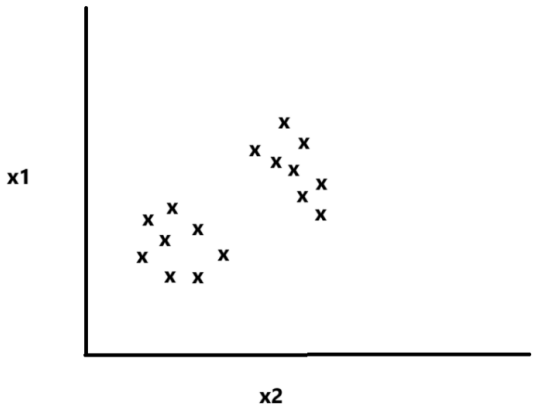
No entraremos en detalle en los fundamentos matemáticos avanzados, ya que:

- No son necesarios para comprender el funcionamiento.
- Pueden dificultar la intuición inicial.

El objetivo es que entiendas **cómo y cuándo usar K-Means**, no solo cómo se calcula.

<u>Ejemplo:</u>

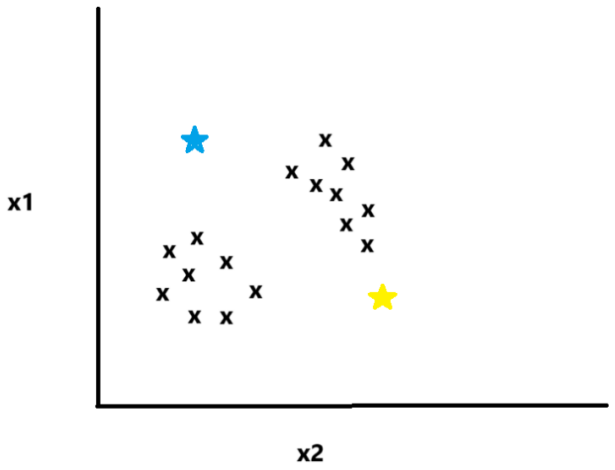
Se dispone del siguiente conjunto de datos sin etiquetar y se desea dividirlo en dos clases que preserven cierta coherencia:



En este caso formado por una variable de entrada x1 y una variable de salida x2 ,es decir, nuestro conjunto de datos está formado por x1 x2.

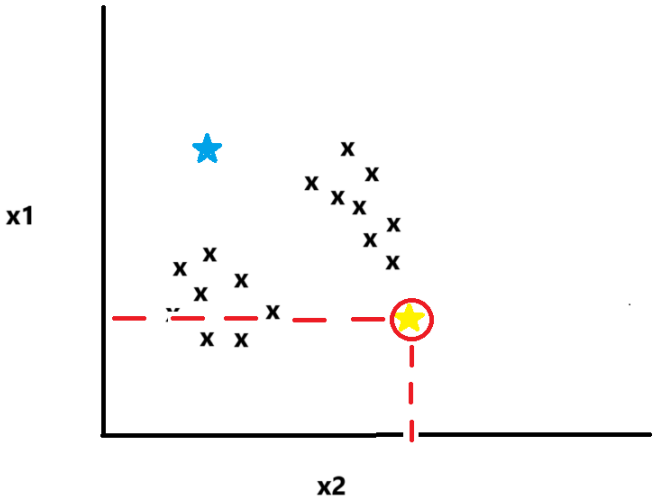
Nosotros tenemos estos datos y lo que queremos es utilizar un algoritmo de Clustering para que de alguna forma, le proporcione una estructura determinada con cierta coherencia. Esta estructura nos podrá ayudar a ganar intuiciones sobre ese conjunto de datos en concreto.

Lo primero que va a hacer el algoritmo KMEANS es inicializar dos puntos de manera aleatoria dentro de nuestro conjunto de datos:



Estos puntos que ves concretamente representados mediante una estrellita azul y una estrellita amarilla se van a denominar **clústers centroides**.

Estos puntos van a tener un valor, podríamos decir por ejemplo que el centroide amarillo tiene estos valores:



De manera más formal vamos a identificar estos puntos con la siguiente notación:

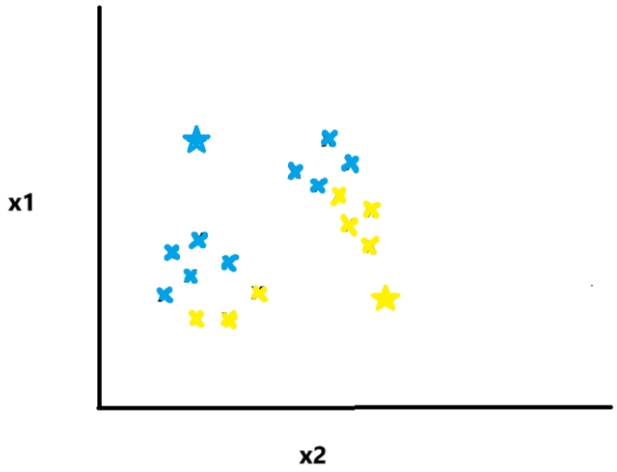
{N1,N2}

En este caso hemos de inicializar dos centroides y los vamos a identificar como N1 y N2. Estos van a ser centroide uno y el centroide dos (estrella azul N1 y la amarilla N2).

Una vez que hemos inicializado de manera aleatoria estos puntos vamos a ver qué continúa haciendo el algoritmo.

Esta es una de las fases fundamentales del algoritmo KMEANS, una vez que hemos inicializado esos puntos centroides lo que va a hacer es realizar una primera agrupación de los datos en función de la distancia. En este caso nuestra métrica de distancia es la distancia euclídea que les separa de cada uno de los centroides que hemos definido.

Cada uno de estos centroides va a formar un clúster o una agrupación, con lo cual nosotros vamos a tener un número de clúster igual al número de centroides(en este caso nosotros os vamos a representar con una C1 y C2).

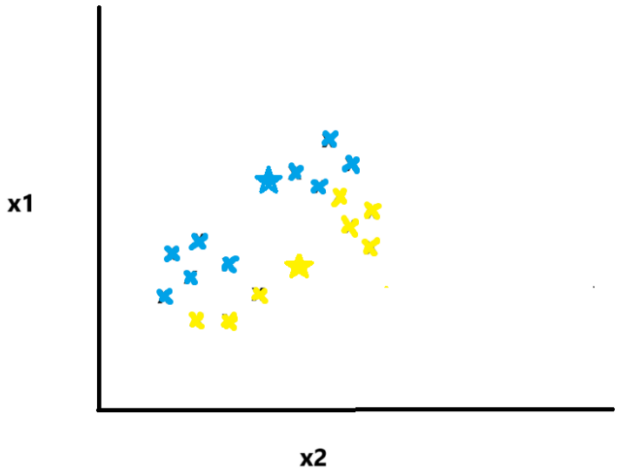


Si nos llegara un ejemplo sin etiquetar, nuestro conjunto de datos está formado por { x1 ,x2 ,…XM}. Este x1 es una forma de representar nuestras dos variables de entrada y que este x1 concretamente se refiere a x1 de 1 y a x2 de 1 porque tenemos un conjunto de datos con dos variables de entrada.

Si la distancia euclídea de nuestro conjunto de datos de entrenamiento a nuestro primer centroide es menor que la distancia de nuestro ejemplo de nuestro conjunto de datos de entrenamiento al segundo centroide se va a asignar a nuestro conjunto 1. Por el contrario si la distancia euclídea de un ejemplo de entrenamiento de nuestro conjunto de datos está o es mayor que la distancia de ese ejemplo al segundo centroide básicamente lo que estamos diciendo es que este ejemplo se va a agrupar en ese segundo clúster.

FUNCIONAMIENTO.

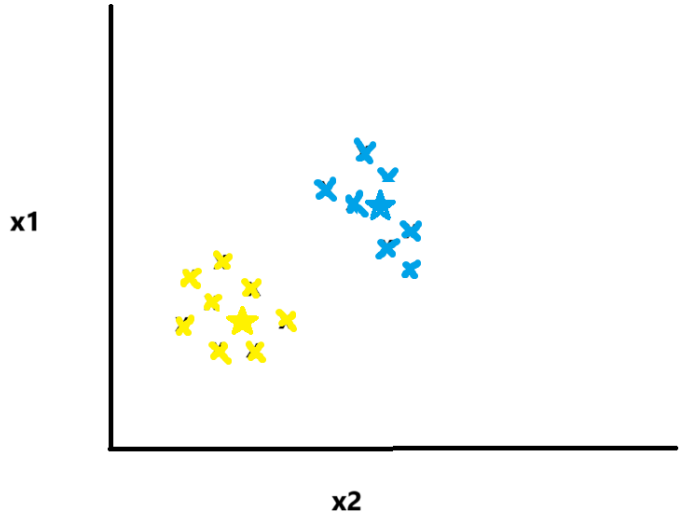
Una vez que tengo todos los ejemplos de mi conjunto de datos de entrenamiento asociados a un clúster (asociados a un centroide, concretamente a ese centroide que está más cercano) ,lo que vamos a hacer es realizar un movimiento de los centroides a un punto con el valor de la media de todos los ejemplos que se han asignado en el conjunto de un clúster determinado. De manera un poquito más gráfica:



Lo que hacemos ahora es mover el centroide al punto medio de los ejemplos asociados a su clase es decir nosotros cogeremos el valor del primer centroide y lo que haremos será modificarlo de manera que va a consistir en 1 partido de la longitud, es decir, el conjunto de ejemplos que hay en ese primer clúster por el sumatorio de todos los ejemplos que hay en ese clúster (calculamos una media aritmética de todos los ejemplos que hay en ese clúster).

El algoritmo sigue repitiendo esto hasta que la posición de los centroides no varíe.

Cuando lleguemos a ese punto en el que los centroides ya no varían, es decir, no varían su localización dentro de nuestro conjunto de datos entonces quiere decir que KMEANS ha terminado de construir esa función hipótesis o ha terminado de construir su modelo. Por lo que los ejemplos de nuestro conjunto de datos de entrenamiento que se encuentran asociados a cada centroide van a quedar así:



Una vez que todos los ejemplos del conjunto de datos han sido asignados a su clúster correspondiente (es decir, al centroide más cercano), el algoritmo pasa a la siguiente fase.

<u>PASO CLAVE: COMO RECALCULAR LOS CENTROIDES.</u>

En este paso, el algoritmo realiza un **ajuste de la posición de los centroides**.

1. ¿Cómo lo hace?

Para cada clúster, calcula la **media de todos los puntos que pertenecen a ese grupo**. Imagina que un clúster contiene varios puntos.

El algoritmo:

1. Suma los valores de todos esos puntos.
2. Calcula la media (promedio).
3. Mueve el centroide a esa nueva posición.

Es decir, el centroide se desplaza al **centro real del grupo de datos**

Por lo tanto, para cada clúster se posiciona de nuevo el centroide con media de todos los puntos asignados a ese clúster. Es decir:

- Se suman todos los ejemplos del clúster.
- Se divide entre el número total de ejemplos.

Esto equivale a calcular una **media aritmética**

2. ¿Qué ocurre después?

Una vez actualizados los centroides: el algoritmo vuelve a repetir todo el proceso:

1. Asignar cada punto al centroide más cercano.
2. Recalcular los centroides.
3. Volver a asignar…

Este proceso se repite hasta que los centroides dejan de moverse. Es decir, llega un punto en el que:

- Las asignaciones ya no cambian.
- Las posiciones de los centroides se mantienen estables.

Cuando esto ocurre, decimos que el algoritmo ha **convergido**. En ese momento:

- El modelo ya está construido.
- Cada dato pertenece a un clúster definido.
- Los centroides representan el centro de cada grupo.

K-Means mejora sus agrupaciones de forma iterativa, moviendo los centroides al centro de los datos hasta encontrar una solución estable.

DBSCAN

Otro de los algoritmos más importantes y utilizados dentro del aprendizaje no supervisado es **DBSCAN** (*Density-Based Spatial Clustering of Applications with Noise*).

DBSCAN es un algoritmo de **clustering no supervisado** que, al igual que K-Means, trabaja con datos sin etiquetar y busca descubrir estructuras dentro de ellos.

Sin embargo, presenta una diferencia fundamental respecto a K-Means:

Mientras que K-Means agrupa los datos en función de la **distancia a un centroide**, DBSCAN utiliza un enfoque distinto: se basa en la **densidad de los datos.**

La densidad hace referencia a zonas donde hay muchos puntos concentrados.

FUNCIONAMIENTO.

El algoritmo analiza el conjunto de datos:

- Identifica regiones donde hay **alta concentración de puntos.**
- Agrupa esos puntos en un mismo clúster.
- Detecta puntos aislados como **ruido o anomalías.**

Una de las grandes ventajas de DBSCAN es que: **No es necesario indicar previamente el número de clústeres.**

El propio algoritmo es capaz de:

- Detectar cuántos grupos existen.
- Adaptarse a la estructura de los datos.

DBSCAN busca:

- Zonas densas → las convierte en clústeres.
- Zonas con pocos puntos → las considera ruido.

Ejemplo.

Imagina un conjunto de datos representado en un plano:

- Hay zonas donde los puntos están muy juntos → clúster.
- Hay puntos aislados → posibles anomalías.

DBSCAN detecta automáticamente estas zonas y construye los grupos.

Pasos: ¿Cómo lo hace DBSCAN?

DBSCAN agrupa puntos según dos ideas:

- **eps**: distancia máxima para considerar que dos puntos están cerca.
- **min_samples**: número mínimo de puntos cercanos para que una zona se considere densa.

DBSCAN clasifica los puntos en:

- **Puntos núcleo**: tienen suficientes vecinos cerca.
- **Puntos borde**: están cerca de un núcleo, pero no tienen suficientes vecinos propios.
- **Ruido**: no pertenecen a ningún grupo.

Ejemplo práctico sencillo:

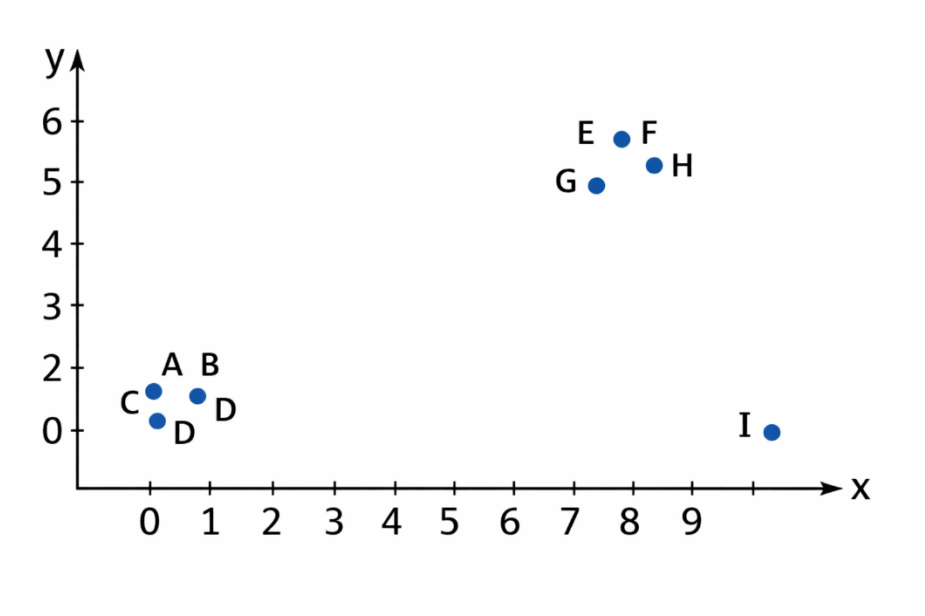
1. **Conjunto de datos.**

Imagina estos puntos en un plano:

- A(1,1)
- B(1.2,1.1)
- C(0.8,0.9)
- D(1.1,0.8)
- E(5,5)
- F(5.2,5.1)

- G(4.9,4.8)
- H(5.1,4.9)
- I(8,1)

Visualmente tendríamos algo así:



Aquí se intuyen:

- Un grupo alrededor de (1,1).
- Otro grupo alrededor de (5,5).
- Un punto aislado en (8,1).

2. Elegimos parámetros.

Supongamos:

- eps = 0.6
- min_samples = 3

Esto significa:

- un punto pertenece al vecindario de otro si está a menos de **0.6 unidades.**
- para ser **núcleo**, un punto debe tener al menos **3 puntos en su vecindad** (según la implementación de scikit-learn, contando el propio punto).

3. Análisis paso a paso.

Punto A(1,1)

Puntos cercanos a A:

- B(1.2,1.1)
- C(0.8,0.9)

- D(1.1,0.8)

Como tiene suficientes vecinos, **A es punto núcleo**.

- **Punto B(1.2,1.1)**

  También está cerca de A, C y D. Por tanto, **B también es núcleo**.

  Lo mismo ocurre con C y D.

  Resultado:

  A, B, C y D forman un **clúster**.

- **Punto E(5,5)**

  Está cerca de:

  - F(5.2,5.1)
  - G(4.9,4.8)
  - H(5.1,4.9)

  También cumple densidad suficiente, así que **E es núcleo**.

  Lo mismo ocurre con F, G y H.

  Resultado:

  E, F, G y H forman otro **cluster**.

- **Punto I(8,1)**

  No tiene vecinos suficientemente cercanos.

  Resultado:

  I es **ruido**.

4. Resultado final.

DBSCAN detecta:

- **Cluster 1**: A, B, C, D.
- **Cluster 2**: E, F, G, H.
- **Ruido**: I.

Limitación.

Si cambias mucho eps:

- sí es demasiado pequeño, muchos puntos pasarán a ser ruido
- sí es demasiado grande, puede unir clústeres que deberían estar separados

Eso ayuda a explicar por qué **la elección de parámetros es importante**.

Aplicación práctica del aprendizaje no supervisado.

Hasta ahora hemos visto la teoría de los algoritmos de aprendizaje no supervisado, como clustering. Sin embargo, en la práctica, aplicar estos algoritmos en un entorno real implica seguir una serie de pasos bien definidos. No se trata simplemente de ejecutar un algoritmo, sino de preparar correctamente los datos, elegir la técnica adecuada y, sobre todo, interpretar los resultados.

Proceso general de aplicación.

Cuando trabajamos con técnicas de clustering en un caso real, normalmente seguimos un flujo estructurado que nos permite obtener resultados útiles y coherentes.

1. Recogida de datos.

El primer paso consiste en obtener los datos que vamos a analizar. En entornos reales, estos datos pueden proceder de múltiples fuentes, por ejemplo:

- Logs de red (registros de actividad de dispositivos)
- Datos de usuarios (comportamiento, accesos, preferencias)
- Tráfico de un servidor (peticiones, paquetes, conexiones)

Es importante destacar que en aprendizaje no supervisado no necesitamos etiquetas, es decir, no sabemos de antemano qué grupo pertenece a cada dato. El objetivo será descubrir esa estructura oculta.

2. Preparación de los datos.

Antes de aplicar cualquier algoritmo, es fundamental realizar un preprocesamiento de los datos. Esta fase es crítica, ya que unos datos mal preparados pueden llevar a resultados incorrectos.

Las tareas principales son:

- **Eliminar valores erróneos o incompletos**, como datos nulos o registros corruptos.
- **Normalizar o escalar los datos**, especialmente importante en algoritmos como K-Means, ya que trabajan con distancias. Si una variable tiene valores mucho mayores que otra, dominará el cálculo.
- **Seleccionar variables relevantes**, eliminando aquellas que no aportan información útil.

Ejemplo didáctico: No tiene sentido comparar directamente una dirección IP con el tamaño de un paquete. Antes, habría que transformar la IP (por ejemplo, a formato numérico o categórico) o descartarla si no aporta valor.

3. Selección del algoritmo.

Una vez preparados los datos, debemos elegir el algoritmo más adecuado en función del problema que queremos resolver.

- **K-Means**
  Se utiliza cuando esperamos encontrar grupos claros, compactos y bien separados. Es eficiente, pero requiere conocer previamente el número de clústeres.

- **DBSCAN**
  Es más flexible, ya que no necesita indicar el número de grupos. Es especialmente útil para:
    - Detectar anomalías .
    - Encontrar estructuras de forma irregular .
    - Identificar ruido en los datos .

En redes, por ejemplo, DBSCAN es muy útil para detectar comportamientos anómalos.

4. Entrenamiento del modelo

En esta fase aplicamos el algoritmo sobre los datos preparados.A diferencia del aprendizaje supervisado:

- No hay etiquetas .
- El modelo **descubre automáticamente patrones** .

El resultado será una asignación de cada dato a un clúster (o a ruido, en el caso de DBSCAN).

5. Interpretación de resultados

Este es uno de los pasos más importantes en aprendizaje no supervisado. El algoritmo:

- Agrupa datos.
- Pero **no explica qué significan esos grupos.**

Por tanto, el analista debe interpretar los resultados.

Ejemplo:
Si obtenemos 3 clústeres en datos de red, podrían corresponder a:

- Usuarios normales .
- Usuarios intensivos .
- Comportamientos anómalos .

Pero esto **no lo dice el algoritmo**, lo deduce el analista.

Optimización de modelos de clustering

Una vez obtenido el modelo, surge una pregunta clave: ¿Cómo sabemos si el modelo está bien ajustado?

Para ello, ajustamos los parámetros del algoritmo.

OPTIMIZACIÓN EN K-MEANS

El parámetro principal es: **Número de clústeres (K).**

Elegir un valor incorrecto puede producir:

- Demasiados grupos (sobreajuste).
- Muy pocos grupos (pérdida de información).

Método del codo:

Se prueba el modelo con distintos valores de K:

- K = 1, 2, 3, 4…

Se mide el error (distancia dentro del clúster).

- Al principio, al aumentar K, el error disminuye mucho.
- A partir de cierto punto, la mejora es mínima.

Ese punto donde la mejora deja de ser significativa se llama "codo".

Ejemplo:

- K = 2 → mejora mucho.
- K = 3 → mejora poco.

Elegimos K = 2.

OPTIMIZACIÓN EN DBSCAN

DBSCAN no usa K, pero tiene otros parámetros:

- **eps** → radio de búsqueda.
- **min_samples** → número mínimo de puntos.

Estos parámetros determinan:

- Cuántos clústeres aparecen.
- Cuántos puntos se consideran ruido.

Ejemplo:

- **eps pequeño** → muchos clústeres pequeños + mucho ruido.
- **eps grande** → pocos clústeres grandes (puede mezclar grupos).

Ajustar estos valores es fundamental para obtener buenos resultados.

Validación en aprendizaje no supervisado

En aprendizaje supervisado **no** sabemos la respuesta correcta porque **NO tenemos etiquetas.**

Validar en aprendizaje no supervisado no consiste en comprobar si es correcto, sino en medir si la estructura encontrada es coherente.

¿Cómo evaluamos el modelo?

Utilizamos **métricas internas**, que analizan:

- Cohesión (qué tan juntos están los datos del mismo clúster).
- Separación (qué tan separados están los clústeres entre sí).

MÉTRICAS PRINCIPALES

Silhouette Score.

Mide qué tan bien está asignado un punto a su clúster.

Valores:

- Cercano a **1** → muy bien agrupado .
- Cercano a **0** → dudoso .
- **Negativo** → mal asignado .

Es una de las métricas más utilizadas en clustering.

Davies-Bouldin Index.

Evalúa la relación entre:

- La dispersión dentro del clúster .
- La separación entre clústeres .

Interpretación:

- **Valor bajo → mejor modelo.**

Interpretación práctica

Un buen modelo de clustering debe cumplir:

- Los datos dentro de cada grupo están **muy próximos entre sí** (alta cohesión) .
- Los grupos están **bien separados entre sí** (alta separación).